

35-Minute Speaker Script for the NDN SF Proposal Defense

Tianxing Ma

PhD Proposal Defense, University of Memphis, June 2026

This document is a concise read-aloud script for the 44-slide proposal deck. Slides 1–34 form the main presentation; Slides 35–44 are backups and references.

Contents

1	Slide 1: Title	3
2	Slide 2: Proposal Defense Roadmap	3
3	Slide 3: Service Framework Problem	3
4	Slide 4: Why NDN for Services	3
5	Slide 5: NDN Packets and Service Data	3
6	Slide 6: Supporting NDN Primitives	3
7	Slide 7: Research Gap	4
8	Slide 8: Related Work: NDN Service Framework Directions	4
9	Slide 9: NDN SF Contribution	4
10	Slide 10: Research Questions	4
11	Slide 11: Completed Foundation: NDN SF v0.1	4
12	Slide 12: NDN SF Entity Roles	4
13	Slide 13: NDN SF Invocation Workflow	5
14	Slide 14: ABE-Backed Authorization: Rationale and Limits	5
15	Slide 15: Runtime Provider Discovery and Selection	5
16	Slide 16: Baselines Before the Comparison	5
17	Slide 17: v0.1 Evidence: Network and Provider Failures	5
18	Slide 18: v0.1 Evidence: Mobility and Availability	6
19	Slide 19: What NDN SF v0.1 Still Lacks	6
20	Slide 20: Post-v0.1 Collaboration Plan Generation	6
21	Slide 21: Proposed Plan-to-Data Enforcement	6
22	Slide 22: UAV Workload and Requirements	6

23 Slide 23: UAV Requirements → NDNF Mechanisms	6
24 Slide 24: Distributed Inference Motivation and Challenge	7
25 Slide 25: DI Requirements → NDNF Mechanisms	7
26 Slide 26: Target DI Architecture	7
27 Slide 27: Inference Plan Generation	7
28 Slide 28: DI Split and Planner Decision	7
29 Slide 29: Preliminary DI Layout Campaign	7
30 Slide 30: Planned DI Validation	8
31 Slide 31: Planned Evaluation Matrix	8
32 Slide 32: Risk, Mitigation, and Timeline	8
33 Slide 33: Expected Contribution and Success Criteria	8
34 Slide 34: Questions?	8
35 Slide 35: Backup: UAV Validation Details	9
36 Slide 36: Backup: Runtime Metadata Evidence	9
37 Slide 37: Backup: User-Side Admission Control	9
38 Slide 38: Backup: NDNF Multi-Provider Coordination	9
39 Slide 39: Backup: Why Not ABE Plus Role-Certificate Authorization?	9
40 Slide 40: Backup: Middleware Comparison	9
41 Slide 41: References [1–7]	9
42 Slide 42: References [8–13]	9
43 Slide 43: References [14–21]	10
44 Slide 44: References [22–25]	10

Slide 1: Title

Good morning. My name is Tianxing Ma, and today I will present my dissertation proposal, *NDNSF: A Data-Centric Service Framework for Secure, Collaborative, and Dynamic Networked Applications*. The dissertation contribution is the NDNSF framework. I use UAV networking and distributed inference as requirement and validation workloads, not as two independent dissertation products. They expose where a service framework must handle mobility, authorization, large named objects, runtime Provider choice, and multi-Provider dependencies.

Slide 2: Proposal Defense Roadmap

The talk has five parts. I will first motivate service frameworks and summarize the NDN mechanisms on which NDNSF builds. I will then define the research gap, contribution, and research questions. Next, I will separate the completed May-20 NDNSF v0.1 baseline from the newer collaboration design and its proposed enforcement mechanism. Finally, I will use UAV and distributed inference to derive requirements, present the evaluation plan, and close with risks and the Spring 2027 timeline.

Slide 3: Service Framework Problem

A service framework turns distributed functions into reusable services by defining discovery, authorization, invocation, selection, failure handling, and data exchange. Existing IP-based frameworks are valuable, but their runtime abstractions are commonly organized around reachable endpoints, channels, registries, and externally configured trust. Applications, however, often care about a named service, an authorized result, and whichever Provider is currently suitable. NDNSF asks whether NDN can make these service and data semantics explicit without requiring every application to rebuild the same coordination logic.

Slide 4: Why NDN for Services

NDN retrieves Data by name rather than by host address. A matching object may come from its producer, an alternate path, a cache, or a repository, and its signature remains verifiable after retrieval. These properties are attractive for mobile Providers, failover, reusable artifacts, and large data. They do not automatically create a service framework. NDNSF adds the missing lifecycle above NDN: who may invoke a named service, which Providers are ready, which one or more are selected, and how the resulting Data drives execution.

Slide 5: NDN Packets and Service Data

At the packet level, an Interest names desired Data, while a Data packet carries a name, metadata, content, and signature. NDNSF does not replace that model. It represents service Requests, ACKs, Selections, Responses, status, and artifact references as named Data. The UAV example makes this concrete: telemetry and video segments have stable names and remain verifiable when retrieved through different paths. The key opportunity is to make service control and service outputs part of the same named-data environment.

Slide 6: Supporting NDN Primitives

NDNSF builds on three supporting primitives. Sync lets participants learn that new named Data exists without a central broker. NAC and NAC-ABE keep cached or repository Data confidential according to name- and attribute-based policy. Trust rules validate which identities may sign particular namespaces. These primitives provide dissemination, content protection, and object authenticity. They still do not determine which runtime Provider holds a collaboration role or whether one valid object follows an authorized dependency edge. That distinction leads to the research gap.

Slide 7: Research Gap

The narrow gap is collaboration-level enforcement for late-bound, multi-Provider execution. NDN can verify that an object is authentic, but authenticity alone does not prove that its signer was assigned the required role or that the object belongs on a declared producer-consumer edge. Existing computation, invocation, and access-control mechanisms provide important ingredients, but applications still need a consistent way to derive concrete roles from runtime willingness and enforce those roles over cacheable intermediate Data. This is the service-layer problem addressed by the revised NDNFSF design.

Slide 8: Related Work: NDN Service Framework Directions

The related work falls into four groups. NFN, NFaaS, Compute First Networking, CaSCON, and data-centric composition address named computation, placement, or dependency graphs. RICE and NSC provide named invocation. NAC, NAC-ABE, Trust Schema, and related systems protect content or constrain valid signers. Other systems focus on measurement, aggregation, or edge security. The claimed gap is deliberately narrower than “service composition”: I have not found a framework that converts an ACK-derived role and dependency plan into Provider-local admission rules for direct or cached intermediate Named Data.

Slide 9: NDNFSF Contribution

The contribution has three connected layers. First, a Request discovers late-bound candidates through signed readiness ACKs and runtime metadata. Second, after ACK collection closes, the newer design assigns only valid candidates to application-defined roles and names every dependency edge. Third, the proposed Plan-to-Data mechanism compiles that concrete plan into local rules over signer identity, producer role, dependency edge, Data name, and key scope. The contribution is therefore the path from runtime discovery to verifiable, cache-transparent execution, not caching, graph planning, or signatures by themselves.

Slide 10: Research Questions

RQ1 identifies the service-layer gap left by existing NDN primitives and service systems. RQ2 asks how NDNFSF should form a role- and dependency-aware plan after ACK collection and enforce it over cacheable Named Data. RQ3 applies the framework to UAV mobility, commands, freshness, media, and multi-drone collaboration. RQ4 applies it to distributed inference artifacts, placement, and dependency execution. Role confinement, edge conformance, direct-versus-cache behavior, security, reliability, latency, and overhead are evaluation dimensions of RQ2.

Slide 11: Completed Foundation: NDNFSF v0.1

This slide establishes the historical boundary. NDNFSF v0.1 is the completed May-20 baseline. It includes the dynamic C++ API and Python bindings, multi-Provider ACK collection, Provider-selection strategies, the NDN-SVS patch, and MiniNDN evaluation. The API fragments on the right are implementation evidence for registering and requesting services. The baseline could discover ready Providers and select one or more of them, but it did not yet assign distinct collaboration roles or enforce named dependencies between those roles.

Slide 12: NDNFSF Entity Roles

NDNFSF has three entity roles. The Service Controller distributes startup permissions, keys, and certificates. A Service User publishes a named Request and coordinates an invocation. Service Providers decide whether they are authorized and ready, publish ACK metadata, and execute only after selection. Runtime Request, ACK, Selection, and Response messages flow directly between users and Providers through

named Data; the controller is not a centralized runtime broker. This separation keeps authorization bootstrap explicit while preserving distributed execution.

Slide 13: NDNSF Invocation Workflow

The logical workflow is Request, ACK, Selection, and Response. A user publishes one signed Request Data object, and Sync announces its name. Authorized Providers fetch it and return ACKs containing readiness information. After the ACK window closes, the user applies a built-in or custom selection policy. Selected Providers execute and publish Responses. The Pub/Sub layer hides the announce-then-fetch mechanics, while optional piggybacking can reduce a fetch exchange without changing the protocol. This workflow is the completed v0.1 foundation on which the newer collaboration plan is built.

Slide 14: ABE-Backed Authorization: Rationale and Limits

ABE protects discovery content for authorized Providers without enumerating their identities; authorized but unselected Providers can still read it. A DKEY aggregates an entity's service permissions while identity-signing credentials are reused. Challenges check access to required decryption capabilities and invocation-state checks reject replay. DNMP instead uses role signing credentials and Trust Schema rules, which can also be paired with encryption. Current NDNSF runtime status validation and key rotation support permission withdrawal, but require propagation and rekeying. This is a fit-for-purpose design choice, not universal superiority.

Slide 15: Runtime Provider Discovery and Selection

Selective ACK turns acknowledgment into a service-level readiness decision. A Provider may report load, queue time, location, capability, model availability, or safety state, or decline the request. The user collects valid ACKs for a fixed window and then applies first-responding, random, all-selected, or application-defined policy. This is more than endpoint discovery because the decision uses current willingness and metadata. It is still Provider selection rather than role-based collaboration: v0.1 could select multiple responders, but it did not define which Provider produces or consumes each dependency.

Slide 16: Baselines Before the Comparison

Before presenting v0.1 results, I make the evaluated semantics explicit. gRPC calls a method through one configured channel and retains native HTTP/2 over TCP recovery. NSC expresses one named call to one executor. NDNSF uses its Request-ACK-Selection-Response workflow. The network-loss comparison aligns topology, path, offered load, service delay, and the baseline measurement contract, but it combines submitted NDNSF values with one new NSC and gRPC run per loss. It is contextual rather than statistically matched. The Provider-failure table still uses the submitted configurations and needs a multi-endpoint or failover gRPC baseline. Security remains unmatched: gRPC is insecure, NSC skips signature verification, and MiniNDN uses a dummy keychain.

Slide 17: v0.1 Evidence: Network and Provider Failures

The left side combines the submitted NDNSF v0.1 values with one new NSC and gRPC run per loss. NDNSF reports 99.83, 99.67, and 89.83 percent at one, three, and ten percent per-link loss; the paper calls the ten-percent value the best measured result. The new NSC run reports 90.17, 74.00, and 34.67 percent, while the new gRPC run reports 100.00, 100.00, and 90.17 percent. The common displayed workload is ten requests per second with six hundred measured requests and five milliseconds of service work; the new baselines use five seconds of warmup, a five-second deadline, and no application retries. Because NDNSF comes from the submitted batch and each baseline condition has only one new run, this is configuration-aligned context without variance or significance claims, not a statistically matched campaign. The Provider-failure table on the right remains submitted v0.1 evidence under different selection semantics.

Slide 18: v0.1 Evidence: Mobility and Availability

The mobility panel is historical v0.1 context, not a fair current superiority claim: its baselines use a single-provider path and different measurement provenance. The current Spec171 all-request controls show that coverage dominates overall success, while the registered ten-seed switching-opportunity holdout supports a narrower tail-latency advantage when a Provider switch is required. A deterministic transition test separately verifies native discovery without a pre-registered Provider-address list, and the six-seed capacity-matched experiment supports Provider-work efficiency. These results do not imply universal mobility superiority; the real-machine availability panel remains a separate v0.1 diagnostic.

Slide 19: What NDNFSF v0.1 Still Lacks

The v0.1 results answer whether NDNFSF can discover readiness and select available Providers. They do not answer how several selected Providers safely complete different parts of one task. There is no concrete role assignment, named dependency graph, or local rule saying which signed intermediate object may trigger the next role. This is a real gap even with a valid request ID: two Providers can participate in the same request, while only one should be authorized to produce a particular dependency. That limitation motivates the post-v0.1 design.

Slide 20: Post-v0.1 Collaboration Plan Generation

An application first provides a role-and-dependency template, either manually or through a domain-specific generator. After the ACK collection window closes, a replaceable planner assigns willing and capable candidates to roles. NDNFSF then validates candidate membership, role coverage, capabilities, the dependency DAG, object names, artifact state, and key scopes. In the UAV example, UAV-1 captures an image, Edge-2 performs detection, and GS-1 performs mapping over two named edges. Only Providers with valid ACKs for this request may appear in the concrete plan.

Slide 21: Proposed Plan-to-Data Enforcement

A collaboration plan matters only if it constrains execution. The proposed mechanism compiles each local plan slice into an admission predicate. A downstream Provider accepts Data only when cryptographic validation succeeds, the request ID matches, the signer is the Provider assigned to the producer role, and the object name and key scope match a declared edge. Thus, an unassigned P3 is rejected even if it uses the same valid request ID, while the assigned P2 object is accepted whether received directly or from a cache. This remains proposed until implementation and negative-test evidence are complete.

Slide 22: UAV Workload and Requirements

The UAV workload now tests whether the generic framework supports mobile, command-sensitive collaboration. Its requirements are changing reachability, command authorization, telemetry freshness, video and recording objects, and multi-drone missions. I am not proposing a complete flight application. I use these requirements to pressure the framework: mobility tests readiness and reselection, commands test authorization and replay protection, media tests immutable large objects, and multi-drone missions test role assignment and dependency admission. The next slide makes this mapping explicit.

Slide 23: UAV Requirements → NDNFSF Mechanisms

Mobility maps to runtime discovery, readiness ACKs, and status-driven reselection. Commands map to Targeted invocation, NAC-ABE, one-time tokens, and replay checks. Telemetry maps to versioned names and freshness or lifecycle checks. Video maps to manifests and segmented retrieval through caches or a Repo. Multi-drone missions map to the newer collaboration plan and proposed Plan-to-Data admission. This table is a requirements and evaluation contract; it is not evidence that every row is complete. Each

mechanism must be tested against its corresponding workload behavior.

Slide 24: Distributed Inference Motivation and Challenge

Distributed inference divides a model into stages or parallel shards, assigns those pieces to Providers, and exchanges intermediate tensors. It stresses NDNF differently from UAV. A valid execution must coordinate model files, runtime software, Provider resources, tensor names, and dependency order. A graph cut alone is not sufficient if the assigned Provider lacks an artifact or if a downstream stage accepts the wrong intermediate object. DI therefore exposes the need to connect late-bound placement, immutable artifacts, and enforceable named dependencies in one collaboration plan.

Slide 25: DI Requirements → NDNF Mechanisms

The application template names roles, services, outputs, constraints, and the dependency graph. ACK-derived Provider views expose resources, load, network metrics, and exact artifact state. The planner creates concrete role assignments and tensor edges. Providers fetch immutable model and input artifacts through manifests and segmented cache or Repo retrieval. A downstream stage then applies the proposed producer-role and edge-admission rule. Authenticated status, bounded replanning, and application-defined compensation are remaining design and evaluation targets, not claims of distributed atomic transactions or already completed recovery.

Slide 26: Target DI Architecture

The top half of the target architecture is planning and commitment. An application template and ACK-closed Provider views feed a planner and validator, producing a concrete Collaboration Plan. The bottom half is Selection-bound execution. Every assigned Provider receives its local plan slice and admission rules; named tensors follow declared edges between Backbone, Head, and Merge roles. Any role may retrieve immutable artifacts from a Repo or cache. Collaboration planning is the newer design, while enforcement of every edge remains proposed until the implementation and tests support that claim.

Slide 27: Inference Plan Generation

Plan generation combines the model graph and application template with Provider views captured after ACK collection closes. A replaceable strategy chooses a feasible split and role assignment. NDNF validates the DAG, capability and artifact constraints, and materializes concrete Provider assignments and named edges. In the example, P2 may produce the Backbone tensor and P4 may consume it only through the declared feature edge. The placement strategy can change by application; NDNF's responsibility is the validated plan contract and the proposed uniform admission rule for direct or cached Data.

Slide 28: DI Split and Planner Decision

Once a plan is valid, the remaining question is which valid placement is efficient. A shared backbone saves repeated computation but may transfer large feature tensors. Replicated backbones use more local compute but exchange smaller outputs. The planner compares candidate layouts using compute cost, intermediate bytes, RTT, bandwidth, and Provider capacity. This optimization is replaceable application policy, not the claimed NDNF novelty. The framework contribution is to carry the selected roles and edges into an executable, enforceable named-data contract.

Slide 29: Preliminary DI Layout Campaign

The preliminary campaign checks the execution harness with a small ONNX NativeTracer workload. Both layouts use the same full-network MiniNDN setup, and each has ten successful runs on the real ONNX path. Single-Provider execution is faster because fixed cross-Provider dependency exchange dominates

for this small model. This negative result is useful: it exposes overhead and confirms that distributed placement is not automatically better. It is harness evidence only, not a claim about real-model generality or proof of the proposed Plan-to-Data enforcement.

Slide 30: Planned DI Validation

Slide 29 shows existing preliminary harness evidence; this slide defines the remaining validation contract. Tests will cover Repo-backed artifact preparation, generated roles and named edges, tensor sizes and segmentation, prefetch timing, and planner decision quality. Plan-to-Data negative tests will preserve a valid request ID while varying signer, producer role, dependency edge, topic, or key scope. Direct and cached retrieval must yield the same accept-or-reject result. Real-model, container, and Provider-failure gates are required before broader DI claims are made.

Slide 31: Planned Evaluation Matrix

This matrix is a proposal commitment, not a table of completed results. RQ1 uses source-backed gap and traceability analysis. RQ2 requires protocol regressions, same-request negative tests, direct-versus-cache tests, matched baselines, and overhead measurement. RQ3 evaluates UAV mobility, authorization, freshness, media, and collaboration through MiniNDN, SITL, and selected real machines. RQ4 evaluates DI artifacts, placement, dependency execution, and Provider collaboration through MiniNDN clusters and real-model containers. Each future claim is tied to an explicit method.

Slide 32: Risk, Mitigation, and Timeline

The first risk is scope, so UAV and DI remain validation workloads rather than full products. Collaboration overhead will be decomposed by protocol phase, while large-object cost will be measured through segments, bytes, and transfer timing. Repeatable MiniNDN and SITL baselines mitigate variable real-machine behavior. The schedule puts Plan-to-Data implementation and matched evaluation in August 2026, UAV validation in September 2026, and real-model DI and failure gates in October 2026. November and December 2026 are reserved for freezing evidence and delivering the full chair draft. Committee review continues from January through March 2027. The defense announcement must be issued at least three weeks in advance, with the defense held by mid-February 2027. Corrections follow, with an internal ProQuest target of March 5, 2027 before the official March 26, 2027 deadline and Spring 2027 graduation.

Slide 33: Expected Contribution and Success Criteria

The expected dissertation contribution has three parts. First, an ACK-derived collaboration plan assigns only valid runtime candidates to roles and names each dependency edge. Second, Provider-local admission accepts intermediate Data only for the current request, assigned producer role, declared edge, and permitted name and key scope, with identical decisions for direct and cached retrieval. Third, the claim must be supported by same-request negative tests, matched baselines, and measured security, latency, message, and cryptographic overhead. NDN SF is the contribution; UAV and distributed inference are the workloads used to validate and refine it.

Slide 34: Questions?

To summarize, NDN SF begins with completed runtime discovery and secure invocation, then asks whether ACK-derived roles and named dependencies can become enforceable Data-plane policy for direct or cached Named Data. That enforcement claim remains to be implemented and validated. Thank you, and I am happy to take questions.

Slide 35: Backup: UAV Validation Details

UAV validation covers the Targeted command lifecycle, typed state freshness, signed segmented media, and mission recovery. Multi-drone scenarios additionally test role assignment and named dependency admission. The goal is to evaluate generic NDN SF mechanisms under mobile, command-sensitive conditions, not to claim a complete UAV application.

Slide 36: Backup: Runtime Metadata Evidence

Runtime metadata changes framework decisions rather than serving as descriptive text. Adaptive admission controls whether a request enters the pipeline under load. Selective ACK exposes current Provider readiness before selection. In the newer design, validated ACK metadata also supplies the planning view used for collaboration-role assignment.

Slide 37: Backup: User-Side Admission Control

User-side admission bounds outstanding requests before they enter the Request-ACK-Selection-Response pipeline. It adapts a local window using latency, timeouts, and queue pressure. This is requester-side overload control and must not be confused with the proposed Provider-local Data-admission rule that checks collaboration roles and dependency edges.

Slide 38: Backup: NDN SF Multi-Provider Coordination

The May-20 baseline already allowed one Request to reach multiple authorized Providers, collect readiness ACKs, and select one or more responders. That is multi-Provider discovery and selection, not yet role-based collaboration. The post-v0.1 design adds explicit roles and named dependencies; the proposed enforcement step converts them into Provider-local rules.

Slide 39: Backup: Why Not ABE Plus Role-Certificate Authorization?

ABE for confidential discovery combined with role-certificate execution authorization is a valid alternative. The reason for retaining ABE-backed challenges is to reuse attribute-based service permission management rather than introduce an additional service-role signing-credential structure. This is not free: Provider DKEYs are needed for protected discovery, while User permission DKEYs and challenge processing are extra authorization costs. A fair comparison holds permissions, confidentiality, replay behavior, and revocation requirements constant, then counts credential issuance, updates, bytes, and processing. We do not claim that a Trust Schema cannot authorize execution or that ABE is universally cheaper.

Slide 40: Backup: Middleware Comparison

gRPC and ROS 2 are useful systems with different runtime abstractions. gRPC provides endpoint-oriented RPC with external resolution, load balancing, and retry facilities. ROS 2 provides a distributed robotics graph. NDN instead makes named, signed, cacheable Data the network object. NDN SF studies the additional service semantics needed above that model; it is not presented as a universal replacement for either middleware.

Slide 41: References [1–7]

This lookup slide contains the foundational references for NDN, Sync, SVS, NAC, NAC-ABE, NFN, and NFaaS. Their numbers appear at the corresponding first mentions in the main presentation.

Slide 42: References [8–13]

This lookup slide covers CFEC, RICE, NSC, MIA-NDN, DNMP, and SECaaS. These sources support the comparison of placement, invocation, measurement, and application-specific service systems.

Slide 43: References [14–21]

This lookup slide covers Compute First Networking, CaSCON, Trust Schema, gRPC, ROS 2, enhanced NAC-ABE, access-controlled in-network processing, and data-centric service composition. Together, they define the boundary around the narrower ACK-derived plan-to-admission contribution.

Slide 44: References [22–25]

This lookup slide provides the first-mention references for Mini-NDN, PX4 SITL, NDN Repo, and ONNX, covering the emulation, flight simulation, repository protocol, and model representation used by the validation workloads.